

StockPilot

Enterprise Inventory & Warehouse Management Platform

A Complete Full-Stack Build Guide

Spring Boot • React • MySQL • Docker

From Empty Folder to Resume-Ready Project

Backend + Frontend + Deployment — Start to End

Scope locked: Product · Warehouse · Supplier · Inventory · Reservation ·

SalesOrder · PurchaseOrder · StockTransfer · AuditLog · Notification

Two roles (ADMIN / STAFF) — no multi-tenant, no refresh tokens

Table of Contents

1. Project Overview & What You're Building
2. Why This Scope (and What We Deliberately Skipped)
3. Tech Stack & Prerequisites
4. Architecture & How the Pieces Fit
5. Data Model & Entity Relationships
6. The 10-Step Roadmap (Overview)
7. Step 1 — Project Setup & Entity Layer
8. Step 2 — Repositories
9. Step 3 — DTOs
10. Step 4 — Core Business Logic (the heart of the project)
11. Step 5 — Supporting Services
12. Step 6 — Security (JWT + Roles)
13. Step 7 — Controllers + Swagger
14. Step 8 — React Frontend
15. Step 9 — Docker Compose
16. Step 10 — Tests & README
17. Day-by-Day Schedule
18. Interview Talking Points
19. Common Pitfalls & How to Avoid Them

1. Project Overview & What You're Building

StockPilot is an internal warehouse operations tool — the kind of software a company like Amazon or Flipkart runs behind the scenes to track what stock sits in which warehouse, reserve items while a customer is paying, move stock between locations, and keep an audit trail of who changed what. It is *not* a customer-facing storefront; there is no shopping cart or checkout for the public. Staff and admins log in, manage inventory, and process orders.

The application is built as two connected halves:

- **Backend** — a Spring Boot REST API that holds all business rules (reservations, transfers, locking) and talks to MySQL.
- **Frontend** — a React single-page app that logs in, shows dashboards and tables, and calls the backend over HTTP.

The whole thing runs with a single `docker compose up`, which is exactly the "it just works" impression you want when demonstrating it in an interview.

The features that make this stand out

Most student inventory projects stop at "product has a stock number, subtract when you buy." StockPilot goes further, and these three pieces are what you should talk about in interviews:

Feature	Why it matters
Reservation system	Stock is <i>reserved</i> during checkout, not immediately deducted. Available = OnHand – Reserved. This mirrors how real commerce works and avoids overselling.
Stock transfer	Moving items from Warehouse Pune to Warehouse Mumbai happens inside one database transaction — either the whole move succeeds or none of it does.
Optimistic locking	Two staff editing the same inventory row at once won't silently overwrite each other, thanks to a <code>@Version</code> field.

2. Why This Scope (and What We Deliberately Skipped)

A bigger feature list is not automatically a better resume project. Every feature you add is something you must build, debug, and be able to explain under questioning. We locked scope to the features that showcase real backend skill without threatening your timeline.

In scope (build these)

Entity / Feature	Purpose
Product (with Category field)	The items being tracked; category adds searchability cheaply.
Warehouse	Physical locations that hold inventory.
Supplier	Who supplied purchased stock — closes an obvious gap on PurchaseOrder.
Inventory	Stock of a product in a warehouse (on-hand + reserved).
Reservation	Temporary hold on stock during an order; expires if unpaid.
SalesOrder	Outgoing order that reserves then confirms stock.
PurchaseOrder	Incoming stock from a supplier.
StockTransfer	Atomic movement of stock between warehouses.
AuditLog	Who changed stock, when, and why.
Notification	Alerts (low stock, transfer complete) — piggybacks on audit trigger points.

Deliberately skipped (and why)

Organization (multi-tenant). This isn't a feature — it's a new dimension that must thread through every query. Miss one org filter and Warehouse A's data leaks into Warehouse B's dashboard. Highest risk-to-reward item in the whole plan.

A third CUSTOMER role. A real customer role implies a browse → cart → checkout storefront, which quietly turns an internal ops tool into an e-commerce site. Pick one identity. We stay with **ADMIN / STAFF**.

A 7-chart dashboard. Each chart is its own aggregation query plus its own component. Two or three clean charts beat seven rushed ones, and interviewers care far more about your reservation/transfer logic than chart count.

Refresh tokens. Real-world correct, but rarely probed at campus level. A single access token keeps auth simple.

3. Tech Stack & Prerequisites

Backend language	Java 17
Backend framework	Spring Boot 3.x (Web, Data JPA, Security, Validation)
Auth	Spring Security + JWT (single access token)
ORM	Hibernate via Spring Data JPA
Database	MySQL 8
Build tool	Maven
Boilerplate reduction	Lombok
API docs	springdoc-openapi (Swagger UI)
Frontend	React 18 + Vite
Styling	Tailwind CSS
HTTP client	Axios
Routing	React Router
Charts	Recharts
Deployment	Docker Compose (Spring Boot + MySQL)

What to install before you start

- **JDK 17** — check with `java -version`
- **Maven** — `mvn -version`
- **Node.js 18+** and npm — `node -v`
- **MySQL 8** (or just run it via Docker later)
- **Docker Desktop** — for the final one-command run
- An IDE: IntelliJ IDEA (backend) + VS Code (frontend) is a common combo

4. Architecture & How the Pieces Fit

StockPilot follows a classic layered architecture. A request flows top to bottom on the backend, and data flows back up:



The golden rule that keeps this clean: **controllers stay thin, services stay smart, repositories stay dumb**. No business

logic in a controller; no business logic in a repository. If an interviewer asks "where does your reservation logic live?", the answer is always "in the service layer, inside a transaction."

Why DTOs matter here

You never return a JPA entity straight out of a controller. Entities carry lazy relationships, internal fields, and the `@Version` column the client shouldn't see or set. Instead you map entity → `Response` DTO on the way out, and `Request` DTO → entity on the way in. This is a small discipline that reads as "this person has worked on a real API."

5. Data Model & Entity Relationships

The core relationship — and the thing that makes the design look professional — is that inventory belongs to a warehouse, not directly to a product:



Key fields per entity

Entity	Important fields
Product	id, sku, name, category, unitPrice, lowStockThreshold
Warehouse	id, code, name, city
Supplier	id, name, email, phone
Inventory	id, product, warehouse, quantityOnHand, quantityReserved, <code>@Version</code>
InventoryReservation	id, inventory, quantity, status, expiresAt, salesOrder
SalesOrder	id, orderNumber, status, items, createdAt
PurchaseOrder	id, orderNumber, supplier, warehouse, status, items
StockTransfer	id, fromWarehouse, toWarehouse, status, items
AuditLog	id, action, entityType, entityId, performedBy, reason, timestamp
Notification	id, type, message, read, createdAt
User	id, username, passwordHash, role (ADMIN/STAFF)

The one formula to remember: `Available = quantityOnHand - quantityReserved`. Everything in the reservation system exists to keep this number honest.

6. The 10-Step Roadmap (Overview)

Build in this order. Each step depends on the one before it, and this sequence means you're always running working code rather than debugging ten unfinished things at once.

Step	What	Why now
1	Project setup + entity layer	Nothing works without the data model.
2	Repositories	Thin data access; two special queries matter.
3	DTOs	Define the API shape before writing services.
4	Core business logic	Reservation + transfer — the centerpiece.
5	Supporting services	CRUD, orders, audit + notification writes.
6	Security (JWT + roles)	Lock down endpoints once they exist.
7	Controllers + Swagger	Expose and document the API.
8	React frontend	Make it demoable.
9	Docker Compose	One-command run.
10	Tests + README	Prove the hard parts; sell the project.

7. Step 1 — Project Setup & Entity Layer

7.1 Generate the Spring Boot project

Go to start.spring.io and select: Maven, Java 17, Spring Boot 3.x. Add dependencies: **Spring Web, Spring Data JPA, Spring Security, Validation, MySQL Driver, Lombok**. Group `com.stockpilot`, artifact `stockpilot`.

pom.xml — extra dependency for Swagger

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.5.0</version>
</dependency>
```

application.yml

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/stockpilot?createDatabaseIfNotExist=true
    username: root
    password: root
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true
  app:
    jwt:
      secret: change-this-to-a-long-random-secret-key-at-least-256-bits
      expiration-ms: 86400000 # 24 hours
```

7.2 Package structure

```
com.stockpilot
├── entity
├── repository
├── dto
│   ├── request
│   └── response
├── service
├── controller
├── security
├── config
└── exception
```

7.3 Enums

```
public enum Role { ADMIN, STAFF }

public enum ReservationStatus { ACTIVE, CONFIRMED, RELEASED, EXPIRED }

public enum OrderStatus { CREATED, RESERVED, CONFIRMED, CANCELLED }

public enum TransferStatus { PENDING, COMPLETED, FAILED }

public enum NotificationType { LOW_STOCK, TRANSFER_COMPLETED, RESERVATION_EXPIRED }
```

7.4 Core entities

Lombok's `@Getter/@Setter` keeps these short. Note the `@Version` field on Inventory — this is optimistic locking, and it's a headline feature.

Product.java

```

@Entity @Getter @Setter
@NoArgsConstructor @AllArgsConstructor @Builder
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String sku;

    @Column(nullable = false)
    private String name;

    private String category;          // Improvement: searchability
    private BigDecimal unitPrice;
    private Integer lowStockThreshold; // triggers LOW_STOCK notification
}

```

Warehouse.java & Supplier.java

```

@Entity @Getter @Setter
@NoArgsConstructor @AllArgsConstructor @Builder
public class Warehouse {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(unique = true) private String code;
    private String name;
    private String city;
}

@Entity @Getter @Setter
@NoArgsConstructor @AllArgsConstructor @Builder
public class Supplier {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String phone;
}

```

Inventory.java — the optimistic-locking core

```

@Entity @Getter @Setter
@NoArgsConstructor @AllArgsConstructor @Builder
@Table(uniqueConstraints = @UniqueConstraint(
    columnNames = {"product_id", "warehouse_id"}))
public class Inventory {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false) private Product product;
    @ManyToOne(optional = false) private Warehouse warehouse;

    private Integer quantityOnHand = 0;
    private Integer quantityReserved = 0;

    @Version                      // OPTIMISTIC LOCK
    private Long version;

    @Transient
    public int getAvailable() {
        return quantityOnHand - quantityReserved;
    }
}

```

InventoryReservation.java

```

@Entity @Getter @Setter
@NoArgsConstructor @AllArgsConstructor @Builder
public class InventoryReservation {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false) private Inventory inventory;
    private Integer quantity;

    @Enumerated(EnumType.STRING)
    private ReservationStatus status;

    private LocalDateTime expiresAt;
    private Long salesOrderId;
}

```

The remaining entities (SalesOrder + SalesOrderItem, PurchaseOrder + PurchaseOrderItem, StockTransfer + StockTransferItem, AuditLog, Notification, User) follow the same pattern: an `@Id`, a status enum, a few fields, and `@ManyToOne` / `@OneToMany` links. Keep order-line items as separate child entities with `@OneToMany(cascade = ALL, orphanRemoval = true)` on the parent.

8. Step 2 — Repositories

Repositories are plain interfaces — Spring Data writes the implementation. Most are one line. Only two carry custom queries that the business logic depends on.

```

public interface ProductRepository extends JpaRepository<Product, Long> {
    Optional<Product> findBySku(String sku);
    List<Product> findByCategory(String category);
}

public interface InventoryRepository extends JpaRepository<Inventory, Long> {
    Optional<Inventory> findByProductIdAndWarehouseId(Long p, Long w);

    // Special query 1: low-stock lookup (drives notifications + dashboard)
    @Query("SELECT i FROM Inventory i WHERE " +
        "(i.quantityOnHand - i.quantityReserved) < :threshold")
    List<Inventory> findLowStock(int threshold);
}

public interface ReservationRepository
    extends JpaRepository<InventoryReservation, Long> {

    // Special query 2: expired reservations for the scheduled job
    List<InventoryReservation> findByStatusAndExpiresAtBefore(
        ReservationStatus status, LocalDateTime now);
}

```

The rest — `WarehouseRepository`, `SupplierRepository`, `SalesOrderRepository`, `PurchaseOrderRepository`, `StockTransferRepository`, `AuditLogRepository`, `NotificationRepository`, `UserRepository` (with `findByUsername`) — are all just `extends JpaRepository<X, Long>`.

9. Step 3 — DTOs

Define the API contract. Requests carry validation annotations; responses expose only what the client needs.

```
// request/ProductRequest.java
public record ProductRequest(
    @NotBlank String sku,
    @NotBlank String name,
    String category,
    @NotNull @Positive BigDecimal unitPrice,
    @NotNull @PositiveOrZero Integer lowStockThreshold
) {}

// response/ProductResponse.java
public record ProductResponse(
    Long id, String sku, String name,
    String category, BigDecimal unitPrice, Integer lowStockThreshold
) {}

// request/ReserveStockRequest.java
public record ReserveStockRequest(
    @NotNull Long productId,
    @NotNull Long warehouseId,
    @NotNull @Positive Integer quantity,
    @NotNull Long salesOrderId
) {}

// response/InventoryResponse.java
public record InventoryResponse(
    Long id, String productName, String warehouseName,
    int onHand, int reserved, int available
) {}
```

Using Java `record` for DTOs is compact and reads as modern Java. Do the entity $\leftrightarrow$ DTO mapping in a small mapper class or inline in the service — no need for MapStruct on a project this size.

10. Step 4 — Core Business Logic (the heart of the project)

This is the section interviewers will dig into. Every method here is transactional, and the reservation/transfer logic is where you demonstrate that you understand concurrency and atomicity.

10.1 The reservation flow

```
Stock = 10
| reserve 2
▼
onHand=10, reserved=2 → available = 8
| payment success → confirm
▼
onHand=8, reserved=0 → available = 8
(or) payment never comes → expiry job releases → reserved back to 0
```

ReservationService.java

```

@Service
@RequiredArgsConstructor
public class ReservationService {

    private final InventoryRepository inventoryRepo;
    private final ReservationRepository reservationRepo;
    private final AuditLogService auditLog;

    @Transactional
    public InventoryReservation reserve(ReserveStockRequest req) {
        Inventory inv = inventoryRepo
            .findByProductIdAndWarehouseId(req.productId(), req.warehouseId())
            .orElseThrow(() -> new NotFoundException("Inventory not found"));

        if (inv.getAvailable() < req.quantity()) {
            throw new InsufficientStockException(
                "Only " + inv.getAvailable() + " available");
        }

        inv.setQuantityReserved(inv.getQuantityReserved() + req.quantity());
        inventoryRepo.save(inv); // @Version guards against lost updates

        InventoryReservation r = InventoryReservation.builder()
            .inventory(inv)
            .quantity(req.quantity())
            .status(ReservationStatus.ACTIVE)
            .expiresAt(LocalDateTime.now().plusMinutes(15))
            .salesOrderId(req.salesOrderId())
            .build();

        auditLog.record("RESERVE", "Inventory", inv.getId(),
            "Reserved " + req.quantity());
        return reservationRepo.save(r);
    }

    @Transactional
    public void confirm(Long reservationId) {
        InventoryReservation r = reservationRepo.findById(reservationId)
            .orElseThrow(() -> new NotFoundException("Reservation not found"));
        Inventory inv = r.getInventory();

        // Reserved stock actually leaves the warehouse
        inv.setQuantityReserved(inv.getQuantityReserved() - r.getQuantity());
        inv.setQuantityOnHand(inv.getQuantityOnHand() - r.getQuantity());
        r.setStatus(ReservationStatus.CONFIRMED);

        inventoryRepo.save(inv);
        reservationRepo.save(r);
        auditLog.record("CONFIRM", "Inventory", inv.getId(),
            "Confirmed " + r.getQuantity());
    }

    @Transactional
    public void release(InventoryReservation r) {
        Inventory inv = r.getInventory();
        inv.setQuantityReserved(inv.getQuantityReserved() - r.getQuantity());
        r.setStatus(ReservationStatus.RELEASED);
        inventoryRepo.save(inv);
        reservationRepo.save(r);
    }
}

```

ReservationExpiryJob.java — scheduled release

```

@Component
@RequiredArgsConstructor
public class ReservationExpiryJob {

    private final ReservationRepository reservationRepo;
    private final ReservationService reservationService;
    private final NotificationService notificationService;

    @Scheduled(fixedRate = 60_000)    // every minute
    @Transactional
    public void expireStaleReservations() {
        var expired = reservationRepo.findByStatusAndExpiresAtBefore(
            ReservationStatus.ACTIVE, LocalDateTime.now());

        for (InventoryReservation r : expired) {
            reservationService.release(r);
            r.setStatus(ReservationStatus.EXPIRED);
            notificationService.push(NotificationType.RESERVATION_EXPIRED,
                "Reservation " + r.getId() + " expired");
        }
    }
}

```

Enable scheduling by adding `@EnableScheduling` to your main application class.

10.2 The atomic stock transfer

This is probably your strongest module. Moving stock from one warehouse to another must be all-or-nothing: if the third item in a five-item transfer fails, the first two must roll back. A single `@Transactional` method gives you exactly that — any exception triggers a full rollback.

StockTransferService.java

```

@Service
@RequiredArgsConstructor
public class StockTransferService {

    private final InventoryRepository inventoryRepo;
    private final StockTransferRepository transferRepo;
    private final NotificationService notificationService;
    private final AuditLogService auditLog;

    @Transactional    // all items move, or none do
    public StockTransfer transfer(StockTransfer transfer) {
        for (StockTransferItem item : transfer.getItems()) {

            Inventory from = inventoryRepo.findByProductIdAndWarehouseId(
                item.getProductId(), transfer.getFromWarehouse().getId())
                .orElseThrow(() -> new NotFoundException("Source inventory"));

            if (from.getAvailable() < item.getQuantity()) {
                throw new InsufficientStockException(
                    "Not enough stock to transfer product "
                    + item.getProductId());    // → rolls back everything
            }
            from.setQuantityOnHand(from.getQuantityOnHand() - item.getQuantity());

            Inventory to = inventoryRepo.findByProductIdAndWarehouseId(
                item.getProductId(), transfer.getToWarehouse().getId())
                .orElseGet(() -> createEmptyInventory(
                    item.getProductId(), transfer.getToWarehouse()));
            to.setQuantityOnHand(to.getQuantityOnHand() + item.getQuantity());

            inventoryRepo.save(from);
            inventoryRepo.save(to);
        }

        transfer.setStatus(TransferStatus.COMPLETED);
        StockTransfer saved = transferRepo.save(transfer);

        auditLog.record("TRANSFER", "StockTransfer", saved.getId(),
            "Transfer " + transfer.getFromWarehouse().getCode()
            + " → " + transfer.getToWarehouse().getCode());
        notificationService.push(NotificationType.TRANSFER_COMPLETED,
            "Transfer #" + saved.getId() + " completed");
        return saved;
    }
}

```

Interview gold: "Why is your transfer safe?" → "The whole loop runs in one transaction. If any item lacks stock, I throw, Spring rolls back the transaction, and none of the earlier deductions persist. There's no partial transfer state in the database."

11. Step 5 — Supporting Services

These are the less glamorous but necessary services: plain CRUD, the order flows, and the audit/notification writers that the core services call.

AuditLogService.java (also writes a Notification on low stock)

```
@Service
@RequiredArgsConstructor
public class AuditLogService {
    private final AuditLogRepository repo;

    public void record(String action, String entityType,
                      Long entityId, String reason) {
        String user = SecurityContextHolder.getContext()
            .getAuthentication().getName();
        repo.save(AuditLog.builder()
            .action(action).entityType(entityType).entityId(entityId)
            .performedBy(user).reason(reason)
            .timestamp(LocalDateTime.now()).build());
    }
}
```

NotificationService.java

```
@Service
@RequiredArgsConstructor
public class NotificationService {
    private final NotificationRepository repo;

    public void push(NotificationType type, String message) {
        repo.save(Notification.builder()
            .type(type).message(message).read(false)
            .createdAt(LocalDateTime.now()).build());
    }

    public List<Notification> unread() {
        return repo.findByReadFalseOrderByCreatedAtDesc();
    }
}
```

LowStockAlertService.java

```
@Service
@RequiredArgsConstructor
public class LowStockAlertService {
    private final InventoryRepository inventoryRepo;
    private final NotificationService notificationService;

    @Scheduled(fixedRate = 300_000) // every 5 min
    public void checkLowStock() {
        inventoryRepo.findLowStock(10).forEach(inv ->
            notificationService.push(NotificationType.LOW_STOCK,
                inv.getProduct().getName() + " low at "
                + inv.getWarehouse().getName()));
    }
}
```

The CRUD services (`ProductService` , `WarehouseService` , `SupplierService`) are straightforward: inject the repository, map DTO to entity, call `save` / `findAll` / `deleteById` . The order services (`SalesOrderService` , `PurchaseOrderService`) create the order, then a sales order calls `ReservationService.reserve(...)` for each line, and a confirmed purchase order increases `quantityOnHand` .

12. Step 6 — Security (JWT + Roles)

One access token, two roles. Admin-only actions (like transfers and deleting products) are protected with method-level security.

JwtUtil.java (essentials)

```

@Component
public class JwtUtil {
    @Value("${app.jwt.secret}")    private String secret;
    @Value("${app.jwt.expiration-ms}") private long expMs;

    public String generate(String username, String role) {
        return Jwts.builder()
            .setSubject(username)
            .claim("role", role)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + expMs))
            .signWith(Keys.hmacShaKeyFor(secret.getBytes()))
            .compact();
    }

    public String getUsername(String token) { /* parse subject */ }
    public boolean isValid(String token)     { /* verify + expiry */ }
}

```

SecurityConfig.java

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity          // enables @PreAuthorize
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtAuthFilter jwtAuthFilter;

    @Bean
    SecurityFilterChain chain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**",
                                "/swagger-ui/**",
                                "/v3/api-docs/**").permitAll()
                .anyRequest().authenticated())
            .addFilterBefore(jwtAuthFilter,
                UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

Protecting an admin-only endpoint

```

@PreAuthorize("hasRole('ADMIN')")
@PostMapping("/api/transfers")
public TransferResponse transfer(@RequestBody TransferRequest req) { ... }

```

Store passwords with `BCryptPasswordEncoder` — never plain text. On register, encode; on login, compare with `matches()`, then issue a JWT.

13. Step 7 — Controllers + Swagger

Controllers are thin: validate input, call a service, map to a response DTO. Here's the shape for products; every other controller mirrors it.

```

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductService service;

    @GetMapping
    public List<ProductResponse> all() { return service.findAll(); }

    @PostMapping
    @PreAuthorize("hasRole('ADMIN')")
    public ProductResponse create(@Valid @RequestBody ProductRequest req) {
        return service.create(req);
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public void delete(@PathVariable Long id) { service.delete(id); }
}

```

Endpoint map

Method + Path	Purpose	Role
POST /api/auth/register	Create user	public
POST /api/auth/login	Get JWT	public
GET/POST /api/products	List / create products	STAFF / ADMIN
GET/POST /api/warehouses	Warehouses	STAFF / ADMIN
GET/POST /api/suppliers	Suppliers	STAFF / ADMIN
GET /api/inventory	Inventory (on-hand/reserved/available)	STAFF
POST /api/reservations	Reserve stock	STAFF
POST /api/sales-orders	Create + reserve order	STAFF
POST /api/transfers	Atomic transfer	ADMIN
GET /api/notifications	Unread alerts	STAFF
GET /api/dashboard/*	Chart aggregations	STAFF

Once the app is running, Swagger UI lives at <http://localhost:8080/swagger-ui.html> — no Postman needed to demo.

Global exception handling

```

@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<?> notFound(NotFoundException e) {
        return ResponseEntity.status(404).body(Map.of("error", e.getMessage()));
    }
    @ExceptionHandler(InsufficientStockException.class)
    public ResponseEntity<?> stock(InsufficientStockException e) {
        return ResponseEntity.status(409).body(Map.of("error", e.getMessage()));
    }
    @ExceptionHandler(OptimisticLockingFailureException.class)
    public ResponseEntity<?> lock(Exception e) {
        return ResponseEntity.status(409).body(
            Map.of("error", "Someone else just updated this. Retry.");
        );
    }
}

```

14. Step 8 — React Frontend

The frontend exists to make the project *demoable*. Keep scope tight: the single most impressive thing you can show live is the reservation flow — reserve stock, watch *available* drop while *on-hand* stays the same, confirm, watch on-hand drop. A bar chart cannot compete with that.

14.1 Design direction

This is an internal ops console, not a marketing site, so the aesthetic should read as calm, dense, and trustworthy — think a control panel, not a landing page. A restrained approach that works well here:

- **Palette:** a slate/ink neutral base (`#1f2933` text on `#f7f9fb`), one cool accent for primary actions (teal `#1f7a8c`), and semantic colors reserved for meaning only — amber for low stock, green for confirmed, red for failed/expired.
- **Type:** one clean sans (Inter) for everything, with tabular numbers for the quantity columns so figures line up.
- **Layout:** a fixed left sidebar for navigation, a top bar with the logged-in user and notification bell, and a content area of tables and cards. Data density is a feature here, not a flaw.
- **Signature element:** the inventory row that visibly splits into *On hand* / *Reserved* / *Available*, with Available color-coded — it makes the whole reservation concept legible at a glance.

14.2 Project setup

```
npm create vite@latest stockpilot-ui -- --template react
cd stockpilot-ui
npm install axios react-router-dom recharts
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

14.3 Folder structure

```
src/
├── api/axios.js           ← axios instance + JWT interceptor
├── auth/AuthContext.jsx  ← stores token + user, login/logout
├── components/           ← Sidebar, Topbar, Table, StatCard
├── pages/
│   ├── Login.jsx
│   ├── Dashboard.jsx
│   ├── Products.jsx
│   ├── Warehouses.jsx
│   ├── Suppliers.jsx
│   ├── Inventory.jsx     ← reserve action lives here
│   ├── SalesOrders.jsx
│   ├── Transfers.jsx
│   └── Notifications.jsx
└── App.jsx               ← routes + protected-route wrapper
```

14.4 The axios instance (JWT on every request)

```
// api/axios.js
import axios from "axios";

const api = axios.create({ baseURL: "http://localhost:8080/api" });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem("token");
      window.location.href = "/login";
    }
    return Promise.reject(err);
  }
);

export default api;
```

14.5 The inventory screen — your signature UI

```
// pages/Inventory.jsx (core idea)
function InventoryTable() {
  const [rows, setRows] = useState([]);

  useEffect(() => { api.get("/inventory").then(r => setRows(r.data)); }, []);

  const reserve = async (row, qty) => {
    await api.post("/reservations", {
      productId: row.productId, warehouseId: row.warehouseId,
      quantity: qty, salesOrderId: row.demoOrderId,
    });
    const fresh = await api.get("/inventory");
    setRows(fresh.data); // available drops, on-hand unchanged
  };

  return (
    <table>
      <thead><tr>
        <th>Product</th><th>Warehouse</th>
        <th>On hand</th><th>Reserved</th><th>Available</th><th></th>
      </tr></thead>
      <tbody>
        {rows.map(r => (
          <tr key={r.id}>
            <td>{r.productName}</td>
            <td>{r.warehouseName}</td>
            <td>{r.onHand}</td>
            <td>{r.reserved}</td>
            <td className={r.available < 10 ? "text-amber-600" : ""}>
              {r.available}
            </td>
            <td><button onClick={() => reserve(r, 1)}>Reserve 1</button></td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

14.6 Dashboard — cap at 2-3 charts

Using Recharts, show exactly three aggregations, each backed by one `/api/dashboard/*` endpoint: **low-stock trend** (line), **orders over time** (line/bar), and **inventory value by warehouse** (bar). Resist adding more — three clean charts read as focused, seven read as padding.

15. Step 9 — Docker Compose

This single feature does a lot of quiet work in interviews: it says "I can package and run a real system." One command starts MySQL and the Spring Boot app together.

Dockerfile (backend)

```
FROM eclipse-temurin:17-jdk AS build
WORKDIR /app
COPY . .
RUN ./mvnw clean package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

docker-compose.yml

```
services:
  mysql:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: stockpilot
    ports: ["3306:3306"]
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 5s
      retries: 10

  backend:
    build: .
    depends_on:
      mysql:
        condition: service_healthy
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/stockpilot
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: root
    ports: ["8080:8080"]
```

Then simply:

```
docker compose up --build
```

The `depends_on ... condition: service_healthy` block prevents the classic "backend starts before MySQL is ready" crash — worth mentioning if asked.

16. Step 10 — Tests & README

16.1 Test only what's hard

Don't test CRUD — nobody is impressed that `save()` saves. Test the two things you'll claim verbally: that reservations respect available stock and that transfers are atomic. Eight to ten sharp tests beat a hundred trivial ones.

```

@SpringBootTest
class ReservationServiceTest {

    @Autowired ReservationService service;
    @Autowired InventoryRepository inventoryRepo;

    @Test
    void reserve_reducesAvailable_notOnHand() {
        // given inventory onHand=10, reserved=0
        service.reserve(new ReserveStockRequest(prodId, whId, 3, orderId));
        Inventory inv = inventoryRepo.findById(invId).orElseThrow();
        assertEquals(10, inv.getQuantityOnHand()); // unchanged
        assertEquals(3, inv.getQuantityReserved());
        assertEquals(7, inv.getAvailable());
    }

    @Test
    void reserve_failsWhenNotEnoughAvailable() {
        assertThrows(InsufficientStockException.class, () ->
            service.reserve(new ReserveStockRequest(prodId, whId, 999, orderId)));
    }
}

@SpringBootTest
class StockTransferServiceTest {
    @Test
    void transfer_rollsBack_whenOneItemFails() {
        // 2-item transfer where the 2nd item exceeds stock
        assertThrows(InsufficientStockException.class,
            () -> service.transfer(badTransfer));
        // assert the FIRST item's stock was NOT deducted
        assertEquals(originalQty, sourceInv.getQuantityOnHand());
    }
}

```

16.2 README that sells the project

Your README is the first thing a recruiter opens. Structure it so the centerpiece is unmissable:

1. **One-line pitch** + a screenshot of the inventory screen.
2. **How to run** — literally just `docker compose up --build`, then the Swagger URL.
3. **The reservation flow**, explained with the on-hand/reserved/available diagram. This is your differentiator — give it real estate.
4. **Architecture diagram** (the layered picture from Section 4).
5. **Tech stack** and a short "what I'd add next" section (honest scope awareness reads as maturity).

17. Day-by-Day Schedule

A realistic ~2-week plan if you can give it a few focused hours a day. Compress or stretch to fit your actual runway.

Day	Focus	Done when...
1	Setup, application.yml, enums, entities	App starts, tables auto-create in MySQL.
2	Repositories + DTOs	Two special queries compile; DTOs defined.
3-4	ReservationService + expiry job	Reserve/confirm/release work; expiry releases stale holds.
5	StockTransferService	Atomic transfer; failed item rolls everything back.
6	Supporting + audit + notification services	CRUD works; audit rows appear; low-stock alerts fire.
7	Security (JWT + roles)	Login returns token; admin-only endpoints blocked for STAFF.
8	Controllers + Swagger + exception handler	All endpoints callable from Swagger UI.
9-10	React: auth, layout, inventory + reserve	Live reservation demo works end-to-end.
11	React: sales orders, transfers, dashboard (3 charts)	Full click-through demo.
12	Docker Compose	<code>docker compose up</code> runs the whole stack.
13	Tests on reservation + transfer	8-10 green tests proving the hard claims.
14	README + polish + screenshots	Resume-ready.

18. Interview Talking Points

Rehearse crisp answers to these — they're what turn a project on your resume into a conversation you win.

Likely question	Your answer, in one breath
"Why reserve instead of just subtracting?"	To avoid overselling during checkout. Stock is held (reserved), not removed, until payment confirms — and released automatically if it doesn't.
"What if two people edit the same inventory at once?"	Optimistic locking via <code>@Version</code> . The second save fails with an optimistic-lock exception instead of silently overwriting; I return a 409 asking the client to retry.
"How is a multi-item transfer safe?"	The whole transfer runs in one <code>@Transactional</code> method. Any shortfall throws, Spring rolls back, and no partial move persists.
"How do expired reservations get cleaned up?"	A <code>@Scheduled</code> job runs every minute, finds ACTIVE reservations past their expiry, releases the reserved quantity, and marks them EXPIRED.
"Why DTOs instead of returning entities?"	To decouple the API from the schema, hide internal fields like the version column, and avoid lazy-loading serialization issues.
"How do you secure admin actions?"	Stateless JWT auth plus method-level <code>@PreAuthorize("hasRole('ADMIN')")</code> on transfer and delete endpoints.

19. Common Pitfalls & How to Avoid Them

Pitfall	Fix
Backend container starts before MySQL is ready	Use the <code>depends_on ... service_healthy</code> healthcheck (Section 15).
Forgetting <code>@EnableScheduling</code>	Add it to the main class or the expiry job never runs.
Returning entities → infinite JSON / lazy errors	Always map to a Response DTO in the service or controller.
CORS errors from the React app	Add a CORS config allowing <code>http://localhost:5173</code> on the backend.
Confirm subtracts on-hand twice	On confirm, reduce <i>both</i> reserved and on-hand once; on reserve, touch only reserved.
Scope creep (adding Organization "just because")	Re-read Section 2. Polish the core; don't dilute the pitch.
Passwords stored in plain text	Always <code>BCryptPasswordEncoder</code> ; never log or return the hash.
Overspending time on charts	Cap at three. A live reservation demo beats any chart.

Final word. The thing that makes StockPilot memorable isn't the number of entities — it's that you can explain, calmly and precisely, how reservations, optimistic locking, and atomic transfers keep the data honest under concurrency. Build the core well, make it run with one command, and be ready to talk through those three ideas. Everything else in this guide serves that goal.